LDscnR-multi

A User Manual

Petri Kemppainen and Frederic Guillaume

Version 0.9.1, for LDscnR 0.9.1

Abstract

This manual explains how to run genome scans that account for linkage disequilibrium (LD) with `LDscnR-multi`, for one data set or for hundreds in a batch. The workflow uses the `LDscnR` R package to group correlated markers into Stage-1 units before testing, to assemble significant units into outlier regions and to compare the number of significant Stage-1 units with a structure-preserving permutation null. The examples use the full three-spined stickleback panel (117 individuals and 790,578 markers) and the full nine-spined stickleback panel (149 individuals and 1,195,557 markers) analysed in the LDscnR manuscript. Two-chromosome subsets of both panels are included for a quick first run.

Contents

1 Introduction **3**
- 1.1 What problem this workflow solves . . . 3
- 1.2 Relationship to `LDscnR` and to the manuscript . . . 3
- 1.3 Reusing previous results . . . 4

2 Key concepts **4**
- 2.1 Names in the code and in the manuscript . . . 4
- 2.2 Stage-1 clustering: units and `core_snp` . . . 4
- 2.3 `ld_w`: local LD support . . . 4
- 2.4 The floor: the minimum size of a tested unit . . . 5
- 2.5 Outlier regions: assembling significant units . . . 5
- 2.6 Permutation nulls and the permutation function . . . 5

3 Installation and setup **5**
- 3.1 The `LDscnR` version pin . . . 6

4 How many markers should be included? **6**

5 Required input files **7**
- 5.1 `input/`: what you provide . . . 7
  - 5.1.1 A real `perm_fun.R` . . . 8
  - 5.1.2 What the input files look like . . . 9
- 5.2 `cache/` and `output/`: what the workflow writes . . . 9

6 Running the workflow **9**

7 The steps in detail **10**
- 7.1 Stage 1: LD-based clustering . . . 10

7.2 LD-decay report . . . 10
7.3 Structure-alignment check . . . 13
7.4 Checking sensitivity to the minimum unit size . . . 14
7.5 Repeating permutation tests at each floor . . . 16
7.6 Association scan with EMMAX . . . 16
7.7 Unit testing and Stage-2 assembly . . . 17
7.8 Manhattan plots . . . 19

8 Batch summary: `run_all()` 22

9 Configuration reference 22

10 Caching and receipts 24

11 Reproducing the manuscript 24

12 Known limitations 25

13 How to cite 25

A Full output directory tree 26

1 Introduction

`LDscnR-multi` is a workflow of R scripts, not a second R package. You install `LDscnR` from GitHub and then use this repository's scripts to analyse one data set or a collection of data set folders. Each data set keeps its own inputs and results, and a batch run also writes a combined summary.

On one core of an Apple M3 Pro laptop (36 GB memory, R 4.6.1), a complete run took about 19 minutes for the three-spined panel (790,578 markers) and about 36 minutes for the nine-spined panel (1,195,557 markers), including the Manhattan plots. Section 6 breaks these times down by step. They are measured examples, not promised run times, because marker density, memory, processor speed, the number of permutations and the plots all matter. The association scan is fast because `emmax_fast()` reuses the fitted relationship model for every marker or unit, whereas the Stage-1 LD calculations and the floor profile take most of the time.

1.1 What problem this workflow solves

A genome scan for markers associated with a phenotype or an environmental gradient faces a structural problem, because markers in LD with each other are not independent tests. If every marker is tested, one underlying signal, such as one causal region, can produce dozens or hundreds of significant markers. Counting them separately exaggerates the number of associated regions and increases the multiple-testing burden.

`LDscnR` answers this with two stages. Stage 1 groups markers into Stage-1 units from genotype LD alone, without the phenotype. Stage 2 assembles the significant Stage-1 units into outlier regions. For each data set, `LDscnR-multi` runs these steps:

- **Stage 1** (`build_stage1()`, Section 7.1) builds the Stage-1 units, the LD-decay fit and the genomic relationship matrix (GRM).
- **Association scan** (`run_stage_A()`, Section 7.6) uses EMMAX (efficient mixed-model association expedited; Kang et al. 2010). EMMAX is a mixed model that accounts for relatedness and population structure through the GRM. The scan has two tests:
  - the *consensus* test (statistic `unit`) fits one association per Stage-1 unit, using a consensus genotype of its markers;
  - the *Simes* test (statistic `simes`) fits one association per marker and combines the marker p-values within each Stage-1 unit with Simes' method.
- **Unit testing and Stage-2 assembly** (`run_stage_B()`, Section 7.7) decides which Stage-1 units are significant and assembles them into outlier regions. It accepts the association scan's results or marker-wise p-values from another method, such as LFMM2 (latent factor mixed model; Caye et al. 2019).

The rest of this manual, which covers the LD-decay report, the structure-alignment check, the floor profile and the Manhattan plots, describes tools that support, calibrate or display these steps.

1.2 Relationship to `LDscnR` and to the manuscript

`LDscnR` supplies the clustering, association and outlier functions, and `LDscnR-multi` organises them into a repeatable workflow for one or many data sets. The example data are the stickleback panels analysed in the LDscnR manuscript (Kemppainen and Guillaume 2026; Section 13). You do not need the manuscript or its analysis scripts to follow this manual. Section 11 describes how `manuscript/` reproduces the manuscript's empirical results with this workflow.

1.3 Reusing previous results

Every step records its inputs and settings and skips the work when nothing has changed (Section 10). This helps both with one data set and with large batches.

2 Key concepts

This section defines the vocabulary used in the rest of the manual. Skip it if you already know what a Stage-1 unit, the floor and a floor profile are.

2.1 Names in the code and in the manuscript

Some function, column and file names in the code predate the manuscript's terms. They are kept so that existing scripts and outputs keep working. Table 1 lists the correspondence, and the prose of this manual uses the manuscript's terms.

Table 1: Names in the code and output files, and the corresponding terms in the manuscript and in this manual.

Code and output files	Manuscript and this manual
`run_stage_A()`	association scan
`run_stage_B()`, `stageB_<engine>/`	unit testing and Stage-2 assembly
statistic `unit`, engine `emmax_unit`	consensus test
statistic `simes`, engine `emmax_simes`	Simes test
`ld_region_rotation()`, `n_rotations`, `region_rotation.rds`	relocation test, number of relocations
`canonical_regions_*.csv`, "canonical floor"	outlier regions at the reported floor
cluster, `unit_id`, `core_snp`	Stage-1 unit, its index and its representative marker
`size_floor`	minimum Stage-1 unit size (the floor)

2.2 Stage-1 clustering: units and `core_snp`

Before any association test, `LDscnR`'s `ld_complexity_reduction()` groups markers into Stage-1 units from genotype LD alone. The setting `cr_rho` (default 0.50) determines an LD cutoff for each chromosome from its fitted decay curve. Clustering first connects markers through above-cutoff LD links, then refines these groups using complete linkage: every pair within a final unit must meet the cutoff. Each unit has one representative marker, its `core_snp`. The representative marker does not depend on the floor, so the floor profile uses it to match units across floors (Section 7.4). The index `unit_id` cannot be used for this, because it counts only the units that reach a given floor.

2.3 `ld_w`: local LD support

`ld_w` measures local LD support. For each marker, it is the median r^2 with neighbouring markers inside a physical window derived from the chromosome's LD-decay fit. The workflow fits $r^2 = b + (c - b)/(1 + a \cdot d)$ and uses the predicted decay rate `a_pred` to set the window at $\rho = 0.95$. The resulting value is reported as `ld_w_095`.

A high value means that the marker is strongly correlated with its local neighbours; a low value means weak local correlation. The value is not a distance, an association test statistic or a p-value. It depends only on the genotypes and is reported alongside association results. The Manhattan plots can show it in place of association p-values or q-values (Section 7.8).

2.4 The floor: the minimum size of a tested unit

Not every Stage-1 unit is tested. The floor (`size_floor`) is the minimum number of markers a unit must contain to be tested, and smaller units get no unit-level p-value. This reduces the number of tests and makes the minimum support of a tested unit explicit. The floor is never below two. Unless it is set explicitly, it is derived from marker density, with one marker of floor for every 250 markers per Mb of assayed sequence, rounded (`size_floor_markers_per_mb` in `DEFAULTS`). Density is the number of markers divided by the summed chromosome spans (`max(Pos) - min(Pos)`, in Mb). Using density reduces dependence on the number of chromosomes analysed, although selecting chromosomes with different marker densities can still change the floor. For these two-chromosome examples, the rounded floor matches that of the full panel. The floor is 8 for the three-spined panel (about 2,080 markers per Mb) and 12 for the nine-spined panel (about 2,960 markers per Mb). Small units can still contain real signals, so raising the floor trades genomic coverage for a smaller set of better-supported tests (Section 7.4).

2.5 Outlier regions: assembling significant units

Stage 2 groups significant Stage-1 units into *outlier regions*. A region may contain one or more significant units and occupies one row of `region_table.csv` (Section 7.7). There are two assembly modes. The default, `stage2_discovered`, repeats `LDscnR`'s clustering on the significant units only, at its own LD threshold. The alternative, `physical`, merges significant units whose spans lie within a set distance of each other. The examples in this manual use the default.

2.6 Permutation nulls and the permutation function

The permutation null repeats the association scan with permuted phenotypes to count how many significant Stage-1 units could arise without the observed association. *How* the phenotype is permuted matters, because shuffling individuals without regard to population structure can break the structure that the mixed model is meant to account for. `LDscnR-multi` therefore requires every data set with an association scan to supply its own `perm_fun.R` (Section 5) instead of assuming that a generic shuffle is safe. The examples use a structure-preserving permutation that reassigns whole populations' values among populations in the same structure group (`perm_within_groups()`), as in the manuscript.

3 Installation and setup

`LDscnR-multi` needs R (tested with 4.6.1) and the packages in Table 2. The listed versions were used for testing. Earlier versions may work but have not been tested. The base packages `parallel`, `stats`, `tools`, `utils` and `grDevices` come with R, and `devtools` is needed only by developers who load an uninstalled `LDscnR` checkout (Section 3.1).

Table 2: Package dependencies.

Package	Tested version	Source
LDscnR	0.9.1 (GitHub tag v0.9.1)	GitHub
data.table	1.18.2	CRAN
digest	0.6.39	CRAN
ggplot2	4.0.3	CRAN
ggrepel	0.9.8	CRAN
patchwork	1.3.2	CRAN
igraph	2.3.1	CRAN
remotes	2.5.0	CRAN
SNPRelate	1.46.0	Bioconductor
gdsfmt	1.48.0	Bioconductor

The first block needs internet access, but later analyses run from local files. The validated LDscnR revision is recorded in R/00_config.R as LDSCNR_PIN$sha.

```
install.packages(c("remotes", "BiocManager", "data.table", "digest",
                   "ggplot2", "ggrepel", "patchwork", "igraph"))
BiocManager::install(c("SNPRelate", "gdsfmt"))
remotes::install_github("genomeinsights/LDscnR@v0.9.1")
```

LDscnR-multi itself is not installed. Download or clone the repository, and from its directory either source its scripts in R

```
library(LDscnR)
for (f in sort(list.files("R", pattern = "\\.R$", full.names = TRUE))) source(f)
run_dataset("examples/3sp_chr1_chr4")
```

or use run_batch.R from the shell, which does both steps and then runs the workflow on one or more data set folders:

```
Rscript run_batch.R examples/3sp_chr1_chr4 examples/9sp_chr19_chr20
Rscript run_batch.R examples/                 # every subfolder with an input/
```

3.1 The LDscnR version pin

The workflow checks the GitHub commit recorded in the installed package, not only its version number, because package code can change without a version bump. If the commit does not match the validated revision, the run stops before it processes a data set. This prevents a large batch from mixing results obtained with different package code. Reinstall the revision shown above when that happens. Developers who work from a local checkout can set LDSCNR_DEV_LOAD_ALL=1, and LDSCNR_REPO to the checkout's path, before using run_batch.R. That mode loads the checkout and checks its source instead. Ordinary users need neither a local LDscnR checkout nor devtools.

4 How many markers should be included?

For most applications, we suggest starting with a well-spread panel of no more than about **one million filtered markers**. This is a practical target, not a biological or software limit. Only Stage-1 units that reach the floor are tested, so adding many densely packed markers can lengthen the LD calculations and the plotting without adding many distinct tests. The nine-spined panel shows that 1.2 million markers are feasible, but it takes longer and uses more memory than the three-spined panel.

If the starting panel is much denser, thin it *before* you build the input files. First apply the study's usual filters for missing data, minor-allele frequency (MAF) and quality. Then use a recorded random seed to sample markers across every chromosome and across short physical bins within chromosomes, so that very dense bins are limited but every bin keeps several markers. Set the bin width and the number of markers per bin from the genome length and the desired total, so that coverage stays broad. Do not select markers by their association with the phenotype, and do not keep only one marker per Stage-1 unit, because that would defeat the unit-level test. Apply the same selected marker IDs, in the same order, to `genotypes.rds`, `map.rds` and any `external_pvalues/` vectors. Record the filters, the seed, the binning rule and the number of markers kept.

Thinning *can* change LD estimates, unit sizes and outlier regions. An outlier region represented by too few remaining markers may disappear, whereas the additional markers of an unthinned panel could have made its LD pattern testable. When possible, therefore, compare the floor profile and the main outlier regions on one representative denser data set before you apply the thinning rule to hundreds of data sets. The one-million target is not proof that denser data cannot reveal additional regions.

5 Required input files

Every data set is one self-contained folder. Nothing about it is registered anywhere else, and passing its path to `run_dataset()` is all that ties it to the workflow.

5.1 `input/`: what you provide

Genotypes and association inputs

File	Needed for	Contents
`genotypes.rds`	All analyses	An $n \times m$ numeric dosage matrix (values from 0 to 2, `NA` allowed), with individuals in rows (row names are individual IDs) and markers in columns (column names are marker IDs).
`map.rds`	All analyses	A `data.table` or `data.frame` with the columns `marker`, `Chr` and `Pos`. `map$marker` must equal `colnames(genotypes)`, in the same order.
`phenotype.rds`	Association scan	A named numeric vector (names are the row names of the genotypes), or a one-column data frame with those row names.
`perm_fun.R`	Association scan	Plain R that defines `perm_fun(b, y)`, which returns one permuted copy of the phenotype `y` for permutation `b`. There is no default (Section 2). The script can use `.dataset_dir`.

Alignment inputs and optional settings

File	Needed for	Contents
`population.rds`	Alignment check	A named vector from individual to population ID.
`structure_group.rds`	optional	A named vector from individual to a coarser sampling group, such as locality or lineage. The alignment check uses these groups; its default is `population.rds`. The example permutation functions also read this file, but other `perm_fun.R` scripts define their own scheme.
`covariate.rds`	optional	A named vector from individual to a structure covariate, such as lineage. When present, the phenotype is replaced by `resid(lm(y ~ covariate))` before EMMAX, and every permuted phenotype is residualised in the same way after `perm_fun()`, as in the manuscript's nine-spined analysis.
`config.R`	optional	Plain R that assigns any `DEFAULTS` name from `R/00_config.R` (Section 9) to override it for this data set only, for example `cr_rho <- 0.35`.

`external_pvalues/` is a second, optional folder next to `input/`. It holds one `.rds` file per extra p-value engine, each a *named* numeric vector (names are `map$marker`). The file name without `.rds` is the engine name, so `external_pvalues/lfmm.rds` is tested as engine `lfmm`. Unit testing does not care whether a p-value comes from the association scan or from another method, as long as there is one p-value per marker.

5.1.1 A real `perm_fun.R`

The three-spined example's permutation function is shown below. The phenotype (ecotype) is constant within each population, so a shuffle of individuals would create variation within populations that never occurs in the real data. This function instead reassigns which population gets which value, among populations from the same locality. `perm_within_groups()` (in `R/01_dataset_io.R`) visits populations and groups in the same order as the manuscript's modules, so the same seed gives the same permuted phenotypes.

```
population <- readRDS(
  file.path(.dataset_dir, "input", "population.rds"))
structure_group <- readRDS(
  file.path(.dataset_dir, "input", "structure_group.rds"))
perm_fun <- function(b, y) {
  perm_within_groups(y, population, structure_group)
}
```

5.1.2 What the input files look like

These are the first rows of the full three-spined panel (117 individuals and 790,578 markers). The two-chromosome example starts with the same Chr1 markers, so its first rows are identical.

```
> g <- readRDS("input/genotypes.rds"); g[1:4, 1:4]
      Chr1:2480 Chr1:2492 Chr1:2493 Chr1:2494
ind_1  0.007751  0.015384  0.058822  1.000001
ind_2  0.003891  0.003891  0.007751  0.999986
ind_3  1.000000  1.000000  1.000014  0.003891
ind_4  0.000244  0.038510  0.003891  1.000000

> m <- readRDS("input/map.rds"); head(m, 3)
     marker  Chr  Pos
1 Chr1:2480 Chr1 2480
2 Chr1:2492 Chr1 2492
3 Chr1:2493 Chr1 2493

> p <- readRDS("input/phenotype.rds"); head(p)
ind_1 ind_2 ind_3 ind_4 ind_5 ind_6
    0     0     0     0     0     0
```

5.2 `cache/` and `output/`: what the workflow writes

Both folders can be deleted and rebuilt by rerunning the workflow, subject to the receipts deciding what needs recomputing (Section 10). `cache/` holds everything that depends only on the genotypes: the genotype file in SNPRelate's GDS (Genomic Data Structure) format, the LD-decay fit, the Stage-1 units and the GRM. There is no `cache/edge_lists/`, because at full-genome density one chromosome's pairwise LD edge list can take several gigabytes. Every step that needs an edge list rebuilds it from the GDS file, one chromosome at a time, and discards it after use. Section 7 describes `output/` step by step, and Appendix A shows the whole tree.

6 Running the workflow

The usual entry point is `run_dataset(dataset_dir)`, which runs every applicable step, in order, for one data set. Start with the two-chromosome examples in `examples/`, which take a few minutes. A full panel takes minutes per step rather than seconds. Table 3 shows the times of a complete run of each full panel from scratch, on one core of an Apple M3 Pro laptop with 36 GB of memory and R 4.6.1. A second call on an unchanged data set is fast (Section 10).

Table 3: Run times in minutes for a complete run from scratch, on one core (`cores = 1`).

Step	**Three-spined**	**Nine-spined**
Stage 1	8.3	14.8
LD-decay report and structure-alignment check	0.4	0.7
Floor profile, including the association scan and unit testing at the reported floor	5.4	14.3
Unit testing of the external engine (LFMM2, three-spined only) and the Manhattan plots	4.6	6.6
Total	18.6	36.4

```
run_dataset("examples/3sp_chr1_chr4")

result <- run_all(c("examples/3sp_chr1_chr4", "examples/9sp_chr19_chr20"))
result$summary            # one row per data set and engine
result$alignment_summary  # one row per data set with input/population.rds
```

`run_dataset()` calls the following steps. Receipts let the analysis steps reuse results when their inputs and settings are unchanged; reports and plots are regenerated.

1. **Stage 1** (Section 7.1) clusters markers by LD from the genotypes alone. It is always called and reuses cached results when its inputs and settings are unchanged.
2. **LD-decay report** (Section 7.2) writes out what Stage 1 computed. It rewrites the report on every run.
3. **Structure-alignment check** (Section 7.3) runs only if both `input/population.rds` and `input/phenotype.rds` exist.
4. **Floor profile** (Section 7.4) runs by default whenever `input/phenotype.rds` exists. It labels each outlier region by whether it is recovered at half and at twice the floor, but it does not change the reported floor.
5. **Association scan** (Section 7.6) runs if a phenotype is present.
6. **Unit testing and Stage-2 assembly** (Section 7.7) runs once for each engine: the consensus and Simes tests of the association scan, and every file in `external_pvalues/`.
7. **Manhattan plots** (Section 7.8) are drawn as two figures with one panel per engine.

7 The steps in detail

The example outputs in this section come from the full three-spined panel unless stated otherwise. The panel was built by `manuscript/build_inputs.R` (Section 11) and analysed with `run_dataset()` and the default settings.

7.1 Stage 1: LD-based clustering

Function: `build_stage1(dataset_dir, cfg)`. **Depends on:** the genotypes and the map, not the phenotype.

Stage 1 is the genotype-only foundation of every later step. It writes the genotypes to a GDS file, fits the LD-decay curve (Section 2.3), computes `ld_w_095` for every marker, runs `LDscnR::ld_complexity_reduction()` to group markers into Stage-1 units (`cr_rho`, default 0.50) and builds the GRM for the association scan. The GRM uses the GCTA estimator (Yang et al. 2011) on the representative marker of every Stage-1 unit, including single-marker units. Stage 1 writes `cache/stage1.rds` and `cache/genotypes.gds`, and every later step reads from them.

7.2 LD-decay report

Function: `report_ld_decay(dataset_dir, cfg)`. **Writes:** `output/ld_decay/`.

The report writes out the LD-decay fit that `build_stage1()` already computed, without recomputing anything. It writes three files:

- `decay_summary.csv`, with one row per chromosome (described below);
- `decay_print.txt`, the console output of `LDscnR`'s `print.ld_decay()`;
- `decay_curves.pdf`, the plot from `LDscnR`'s `plot.ld_decay()`, with one page for the whole genome and then one page per chromosome (Figure 1).

`decay_summary.csv`: one row per chromosome

The full three-spined panel has 20 rows, one per chromosome. The first four are shown:

```
  Chr chr_size n_snp_chr bp_per_snp            a         c          b
Chr1 28185489     56654   497.4672 0.0023137538 0.9002370 0.05513295
Chr2 23295619     45981   506.6435 0.0025510668 0.9284319 0.05513295
Chr3 16797709     34876   481.5797 0.0032481732 0.9167770 0.05513295
Chr4 32632357     63953   510.2515 0.0009673896 0.8995031 0.05513295
  Chr      a_pred   c_pred n_w_used rho_slide_raw rho_slide_pred
Chr1 0.002095406 0.917752       39     0.9982654      0.9980850
Chr2 0.002464324 0.917752       39     0.9984550      0.9984007
Chr3 0.003255176 0.917752       39     0.9987231      0.9987258
Chr4 0.001849774 0.917752       39     0.9959646      0.9978855
... plus slide_bp_rho_<X> and slide_snp_rho_<X> for X in rho_targets
```

Column	Meaning
`Chr`	Chromosome.
`chr_size`	Chromosome span in base pairs (`max(Pos)`).
`n_snp_chr`	Number of markers on the chromosome.
`bp_per_snp`	Average marker spacing, `chr_size / (n_snp_chr - 1)`, used to convert marker counts to base pairs.
`a`, `c`, `b`	The chromosome's own fitted parameters of the decay curve $r^2 = b + (c - b)/(1 + a \cdot d)$. `a` is the decay rate (larger means faster decay), `c` the short-range level and `b` the genome-wide background LD, which is the same in every row.
`a_pred`, `c_pred`	The values predicted from chromosome size by a robust regression of $\log(a)$ on $\log$(chromosome size) across chromosomes. `ld_w_095` and the size-predicted curve in `decay_curves.pdf` use these values rather than the chromosome's own, noisier `a` and `c`. With fewer than three distinct chromosome sizes, as in the two-chromosome examples, the regression cannot be fitted and the median of `a` is used instead.
`n_w_used`	The number of windows on the chromosome that passed the contrast-over-background check and were used in the fit.
`rho_slide_raw`, `rho_slide_pred`	The share of LD (ρ) retained by the current `slide` setting, computed with `a` and with `a_pred`, respectively.
`slide_bp_rho_<X>`, `slide_snp_rho_<X>`	For each target in `rho_targets` (default 0.99), the window in base pairs and in markers needed to retain that share of LD, computed with `a_pred`.

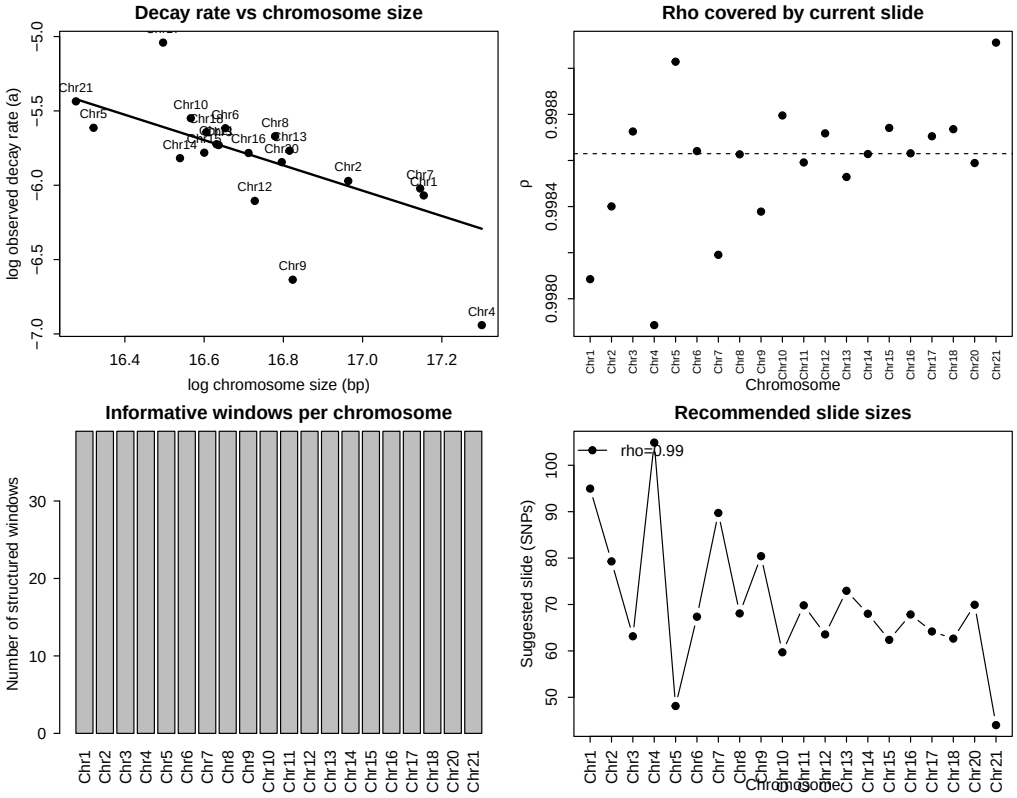


Figure 1: The first page of `decay_curves.pdf` (the whole genome) for the full three-spined panel. It shows the decay rate against chromosome size, with the regression across all 20 chromosomes, the share of LD retained by the current `slide` setting and the informative windows per chromosome. Each chromosome then gets its own page (not shown) with its window-wise fits.

7.3 Structure-alignment check

Function: `check_structure_alignment(dataset_dir, cfg)`. **Runs when:** both `input/population.rds` and `input/phenotype.rds` exist. **Writes:** `output/alignment/`.

Before any association test, the check asks how much of the phenotype is already explained by population structure. It looks at (a) the structure groups in `structure_group.rds`, within which the permutation null shuffles, and (b) the leading axes of the GRM computed between populations (`n_axes`, default five). **It is a warning, not a gate.** A low alignment does not validate a result, and a high alignment does not invalidate one, but a high alignment means that phenotype and ancestry offer nearly interchangeable explanations for any signal the association scan finds. This is worth knowing before an outlier region is interpreted. The check uses the raw phenotype, not the phenotype adjusted for `covariate.rds`.

`observed.csv`: one row

```
n_individuals n_populations n_structure_groups grm_axes_r2_population
          117            35                 4              0.1034668
grm_axes_adjusted_r2_population structure_r2_population
                    -0.05110793               0.0541958
structure_r2_individual_weighted
                       0.1113596
```

Column	Meaning
`n_individuals`, `n_populations`, `n_structure_groups`	The numbers of individuals, populations and structure groups.
`grm_axes_r2_population`	R^2 of the population-mean phenotype regressed on the top eigenvectors of the between-population GRM.
`grm_axes_adjusted_r2_population`	The same, as adjusted R^2, which accounts for the number of axes.
`structure_r2_population`	R^2 of the population-mean phenotype regressed on the structure group, with one point per population.
`structure_r2_individual_weighted`	The same with one point per individual, so populations with more individuals count more.

`null_draws.csv`: one row per null scheme and draw

Each draw (`n_draws`, default 1,000) recomputes the three measures of `observed.csv` on a permuted phenotype or a surrogate. There are two null schemes. The `"within-group permutation"` scheme shuffles within structure groups, and the `"GRM-matched continuous"` scheme draws a continuous surrogate with the covariance of the GRM. The first three draws:

```
                  scheme  draw grm_axes_r2_population structure_r2_population
within-group permutation     1             0.09176547               0.0541958
within-group permutation     2             0.11844692               0.0541958
within-group permutation     3             0.15222279               0.0541958
structure_r2_individual_weighted
                       0.0362699
                       0.2276722
                       0.1280067
```

`null_summary.csv`: one row per null scheme and measure

Each observed measure is compared with its null distribution:

```
                  scheme                  metric  observed null_mean
within-group permutation  grm_axes_r2_population 0.1034668 0.1300878
  GRM-matched continuous  grm_axes_r2_population 0.1034668 0.3039329
within-group permutation structure_r2_population 0.0541958 0.0541958
null_median      null_sd  null_q025 null_q975 observed_within_null95
  0.1199809 5.714173e-02 0.05005395 0.2674474                   TRUE
  0.2901448 1.438419e-01 0.07708650 0.6040352                   TRUE
  0.0541958 1.039326e-16 0.05419580 0.0541958                   TRUE
proportion_null_ge_observed
                      0.628
                      0.938
                      1.000
```

Column	Meaning
`scheme`, `metric`	The null scheme and the measure.
`observed`	The observed value from `observed.csv`.
`null_mean`, `null_median`, `null_sd`	Summary of the null distribution.
`null_q025`, `null_q975`	The 2.5th and 97.5th percentiles of the null.
`observed_within_null95`	`TRUE` if the observed value lies within the central 95% of simulated values. This flag alone does not establish that the observed and null processes are equivalent.
`proportion_null_ge_observed`	The fraction of null draws at least as large as the observed value, a one-sided tail proportion. Low values mean that the observed alignment is unusually strong.

7.4 Checking sensitivity to the minimum unit size

Function: `run_floor_profile(dataset_dir, factors = c(0.5, 2))`. **Runs when:** `input/phenotype.rds` exists, by default (`floor_profile = TRUE` in `run_dataset()`). **Writes:** `output/floor_profile/`.

Why the floor profile exists

Any single floor (Section 2.4) is a practical choice, so the floor profile asks how robust each outlier region is to that choice. Every outlier region found at the reported floor is checked at half and at twice that floor (4 and 16 for a floor of 8). It is labelled `"3/3"` if it is recovered at both comparison floors, `"2/3"` if it is recovered at one and `"1/3"` if it is recovered at neither. The low comparison floor may be 1 (single markers included), but it is never the reported floor.

In the manuscript's simulations (600 runs with known causal variants and a floor of two), the precision of outlier regions was about 0.2 for regions labelled 1/3, 0.5 for 2/3 and 0.8 for 3/3. These values held for the consensus test, the Simes test and the Simes test on LFMM2 p-values, although LFMM2 was somewhat lower (0.44 and 0.75 for 2/3 and 3/3). Dropping 1/3 regions cost little recall. Recovery at the *lower* floor, where more units are tested and the Benjamini-Hochberg (BH) threshold is harder to reach, was the more informative of the two checks.

How recovery is decided

A region is recovered at a comparison floor when at least one of its significant Stage-1 units is also significant at that floor *and* the region overlaps an outlier region assembled at that floor. This is the manuscript's rule. Units are matched across floors by their representative marker (`core_snp`, Section 2), because `unit_id` only indexes the units that reach one floor and two different units can both have the same `unit_id` at two floors. A unit smaller than a comparison floor is not tested there, so it cannot be recovered there.

Only the reported floor gets permutations

- The **reported** floor runs the association scan and unit testing exactly as `run_dataset()` does, with the permutation null, Stage-2 assembly and, if an `annotation` is given, the relocation test.
- The two **comparison** floors are tested on the observed data only, using `observed_only = TRUE` in `run_stage_B()`. This mode never loads a permutation file. The consensus test is fitted once more, at the low floor (`run_stage_A(permute = FALSE)`). The units of the high floor are a subset of those of the low floor, and a unit's EMMAX p-value does not depend on which other units are tested. The high floor therefore reuses those p-values, and only the BH correction and Stage-2 assembly are repeated. The Simes test reuses the marker-wise scan at every floor.

Output files

`grid.csv` has one row per floor. `achieved_reduction` is $1 - n_{\text{units tested}}/n_{\text{markers}}$: the reduction in test count relative to testing every marker. `marker_coverage` is the fraction of all markers that belong to units reaching that floor. For the full three-spined panel (floor 8):

```
    role floor achieved_reduction n_units_tested marker_coverage
     low     4          0.9832806          13218     0.091436898
reported     8          0.9982848           1356     0.021471632
    high    16          0.9997230            219     0.007480603
```

`summary.csv` has one row per test (`unit` for consensus, `simes` for Simes):

```
method floor_low floor floor_high reduction_low reduction reduction_high
  unit         4     8         16     0.9832806 0.9982848       0.999723
 simes         4     8         16     0.9832806 0.9982848       0.999723
n_units_tested n_significant n_regions n_tier_1 n_tier_2 n_tier_3
          1356            97        39       25       10        4
          1356            74        35       23        7        5
```

`canonical_regions_unit.csv` and `canonical_regions_simes.csv` have one row per outlier region at the reported floor. Regions found only at a comparison floor are not listed, because the profile describes how robust the reported regions are. The consensus file of the full three-spined panel has 39 rows, of which 4 are labelled 3/3, 10 are labelled 2/3 and 25 are labelled 1/3. These are the counts reported in the manuscript. The first three rows:

```
             region_id    Chr     from       to tick_004 tick_016 n_ticks
Chr1:21492308-21938477   Chr1 21492308 21938477     TRUE     TRUE       3
Chr4:22225174-22511008   Chr4 22225174 22511008     TRUE    FALSE       2
Chr4:26276344-26545393   Chr4 26276344 26545393    FALSE    FALSE       1
stability_tier   span
           3/3 446169
           2/3 285834
           1/3 269049
```

Column	Meaning
region_id, Chr, from, to, span	The outlier region and its span in base pairs.
tick_<floor>	TRUE if the region is recovered at that comparison floor, by the rule above.
n_ticks, stability_tier	The number of floors, out of three, at which the region is found, as a number and as "3/3", "2/3" or "1/3". It is never "0/3", because every row is a region found at the reported floor.

The tier is a robustness label, not a measure of significance. It is not a p-value, a false discovery rate (FDR) or an independent replication. Every floor analyses the same data, only the reported floor gets the structure-preserving permutation null, and a region that persists across floors can still be caused by population structure. The simulated precisions make the label useful for ranking regions, but they are not a calibrated probability that a region is true in a new data set.

7.5 Repeating permutation tests at each floor

Function: `run_floor_sweep(dataset_dir, factors = c(0.5, 1, 2))`. `run_dataset()` does not run it.

The floor profile keeps two of its three floors cheap by testing them on the observed data only. `run_floor_sweep()` instead reruns the full permutations at half, one and two times the floor. Each floor is at least two, and factors that give the same floor are run once. The sweep estimates a permutation null at every floor, so it costs more than the floor profile. Call `run_floor_sweep()` separately when these additional null comparisons are needed.

`floor_sweep_summary.csv` has one row per floor and test. For the two-chromosome three-spined example (floor 8):

floor	statistic	n_units_tested	n_significant	n_regions
4	unit	2432	17	8
4	simes	2432	9	6
8	unit	340	58	13
8	simes	340	36	8
16	unit	84	42	6
16	simes	84	36	4

Column	Meaning
floor	The floor of this row.
statistic	"unit" (consensus) or "simes".
n_units_tested	The number of units that reached the floor and were tested.
n_significant	The number of those that were significant at alpha.
n_regions	The number of outlier regions assembled from the significant units.

7.6 Association scan with EMMAX

Function: `run_stage_A(dataset_dir, cfg)`. **Writes:** `output/pvalues/emmax_unit/` and `output/pvalues/emmax_simes/`.

Both tests use the same EMMAX mixed model, with the genotypes, the phenotype and the GRM from Stage 1, but they test different things:

- The **consensus test** (`unit`) fits one association per Stage-1 unit that reaches the floor, using a consensus genotype of the unit's markers (`unit_repr = "consensus_dosage"`). It writes `p_obs.rds` and `p_perm.rds`, with one value per tested unit, and `p_obs_marker.rds`.

The last is a separate marker-wise EMMAX scan, used only so that the Manhattan plot shows variation between markers rather than one flat value per unit.

- The **Simes test** (`simes`) fits one association per marker and combines the p-values within each unit that reaches the floor with Simes' method. It writes `p_obs.rds` for *every* marker, which the plots need, and `p_perm_compact.rds`, the permutations for the markers in tested units only. In the full three-spined panel only about 2.1% of the 790,578 markers belong to units of at least eight markers, so restricting the permutations saves most of their cost. The results are identical to an unrestricted scan. `run_stage_A()` takes `simes_floor` (by default the floor), and a later call with a lower floor rebuilds this scan.

With `input/covariate.rds`, the phenotype and every permuted phenotype are residualised on the covariate before EMMAX. Both folders also hold `params.rds` and `_receipt.rds`. The outputs are R objects, and Section 7.7 shows what they become after unit testing.

7.7 Unit testing and Stage-2 assembly

Function: `run_stage_B()`, once per engine, called as `run_stage_B(dataset_dir, engine, cfg = cfg)`. **Writes:** `output/stageB_<engine>/`.

This step turns p-values into decisions. It decides which Stage-1 units are significant after BH correction and assembles them into outlier regions. It also runs a permutation test when permutation p-values are available. This compares the observed number of significant units with the counts for permuted phenotypes. An external file of observed marker p-values alone does not supply a permutation null; pass permutation p-values to `run_stage_B()` to test that engine against its own null. If an annotation is given, this step also runs the relocation test. The relocation test moves the outlier regions to random positions within their chromosomes (10,000 relocations) and asks whether they overlap the annotation more often than expected. Unit testing and Stage-2 assembly use the same procedure for every engine, whether it is the consensus or Simes test of the association scan or a file in `external_pvalues/`. The full three-spined panel has an `external_pvalues/lfmm.rds`, so unit testing runs three times (`emmax_unit`, `emmax_simes` and `lfmm`). It gives 50 outlier regions for LFMM2, compared with 39 for the consensus test and 35 for the Simes test, with the same columns and meaning.

`snp_results.csv`: one row per marker in the map

Most markers are not tested, so the first rows of any panel are mostly `NA`. The rows for markers belonging to significant Stage-1 units (`tested == TRUE & significant == TRUE`) are more informative. The first two from the consensus test:

```
       marker    Chr      Pos statistic marker_p marker_q      unit_p
Chr1:19374296   Chr1 19374296      unit       NA       NA 0.002306879
Chr1:19374302   Chr1 19374302      unit       NA       NA 0.002306879
    unit_q  p_display q_display unit_id              region_id significant
0.03475698 0.10010587 0.8743987      48 Chr1:19374296-19374576        TRUE
0.03475698 0.04834741 0.8170895      48 Chr1:19374296-19374576        TRUE
tested   ld_w_095
  TRUE 0.03257659
  TRUE 0.02403367
```

Column	Meaning
`marker`	Marker ID, from `map$marker`.
`Chr`, `Pos`	Chromosome and position, from the map.
`statistic`	`"unit"` (consensus) or `"simes"`, the test that produced the row.
`marker_p`, `marker_q`	The marker's own p-value and BH q-value. They are defined only for marker-wise p-values (the Simes test or an external engine), and are `NA` for the consensus test, which has no marker-wise p-value.
`unit_p`, `unit_q`	The p-value and q-value of the marker's unit, copied to every member marker. Significance is decided from these, for both tests.
`p_display`, `q_display`	The values the Manhattan plot shows (Section 7.8). They equal `marker_p` and `marker_q` when those exist. For the consensus test they come from the separate marker-wise scan (`p_obs_marker.rds`, Section 7.6), so the plot shows variation between markers rather than one flat value per unit.
`unit_id`	The marker's tested Stage-1 unit, or `NA` if its unit is smaller than the floor and was not tested.
`region_id`	The marker's outlier region, as `"<Chr>:<from>-<to>"`, or `NA` if its unit was not significant.
`significant`	Whether the marker's unit is significant at `alpha` after BH correction. `NA` when `tested` is `FALSE`.
`tested`	Whether the marker's unit reached the floor and was tested, which is the same as `!is.na(unit_id)`.
`ld_w_095`	Local LD support at $\rho = 0.95$ (Section 2.3). It depends only on the genotypes, so it is the same in every engine's file.

Markers are assigned to `unit_id` and `region_id` by Stage-1 and Stage-2 membership, not by coordinates. Stage-1 units and outlier regions are not always physically contiguous, so a region's span can contain markers that belong to another unit or region.

`region_table.csv`: one row per outlier region

The first three outlier regions of the consensus test:

```
   Chr     from       to n_units n_markers occupancy
  Chr1 19374296 19374576       1        10 0.5263158
  Chr1 21492308 21938477      46      1378 0.5529695
 Chr11  2035533  2035916       1         9 0.3103448
```

Column	Meaning
`Chr`, `from`, `to`	The region's span in base pairs.
`n_units`	The number of significant units assembled into the region.
`n_markers`	The number of markers in those units.
`occupancy`	`n_markers` divided by the number of markers in the map between `from` and `to`. A value of 1 means every marker in the span belongs to the region. Lower values mean that other markers are interleaved, so the signal is less contiguous.

`report.txt`

`report.txt` saves the console summaries without changes. It includes the unit-testing summary, the permutation-test summary when available, and the relocation-test summary when available. For the consensus test of the full three-spined panel:

```
<ld_outlier_test> unit / stage2_discovered | floor = 8, alpha = 0.05 | 97/1356 units -> 39 regions
<ld_outlier_perm> observed 97 | surrogate mean 3.48 | p = 0.0080 | null/obs ratio 3.6%
<ld_region_rotation> observed 13/39 | null mean 4.45 | fold 2.92x | p = 0.0002
```

The nine-spined panel shows a useful contrast. Of its 2,784 tested units, 54 are significant after BH correction, but the permutation test is not significant (null mean 49.34, p = 0.261). The phenotype is closely aligned with lineage, so the permuted phenotypes also give many significant units. Many units that are significant after BH correction therefore do not imply that the panel-wide permutation test is significant, because the two tests ask different questions.

```
<ld_outlier_test> unit / stage2_discovered | floor = 12, alpha = 0.05 | 54/2784 units -> 20 regions
<ld_outlier_perm> observed 54 | surrogate mean 49.34 | p = 0.2607 | null/obs ratio 91.4%
```

7.8 Manhattan plots

Function: `ldm_manhattan(dataset_dir, engines, value)`. **Writes:** `output/figures/`.

The Manhattan plots show $-\log_{10}(q)$ (`value = "q"`), $-\log_{10}(p)$ (`value = "p"`) or untransformed local LD support (`value = "ld_w_095"`). The association axes use the marker display values `q_display` and `p_display`, while significance and region membership are decided from unit-level q-values. A coloured marker can therefore have a display q-value above the unit-testing threshold. The plot of `ld_w_095` lets you compare an outlier region with the local LD pattern, but it is not a second association test.

Each engine gets one panel, and all panels share one continuous genome-wide axis, with chromosomes placed end to end and marked by alternating light-grey bands. Markers outside outlier regions form a grey background, and markers in outlier regions are coloured by region. Nearby regions get contrasting colours. The strongest regions in each panel are labelled directly rather than through a legend. When a floor profile (Section 7.4) exists for a panel's test, the label also shows the tier, for example `"R5 (3/3)"`. External engines have no floor profile and keep plain labels.

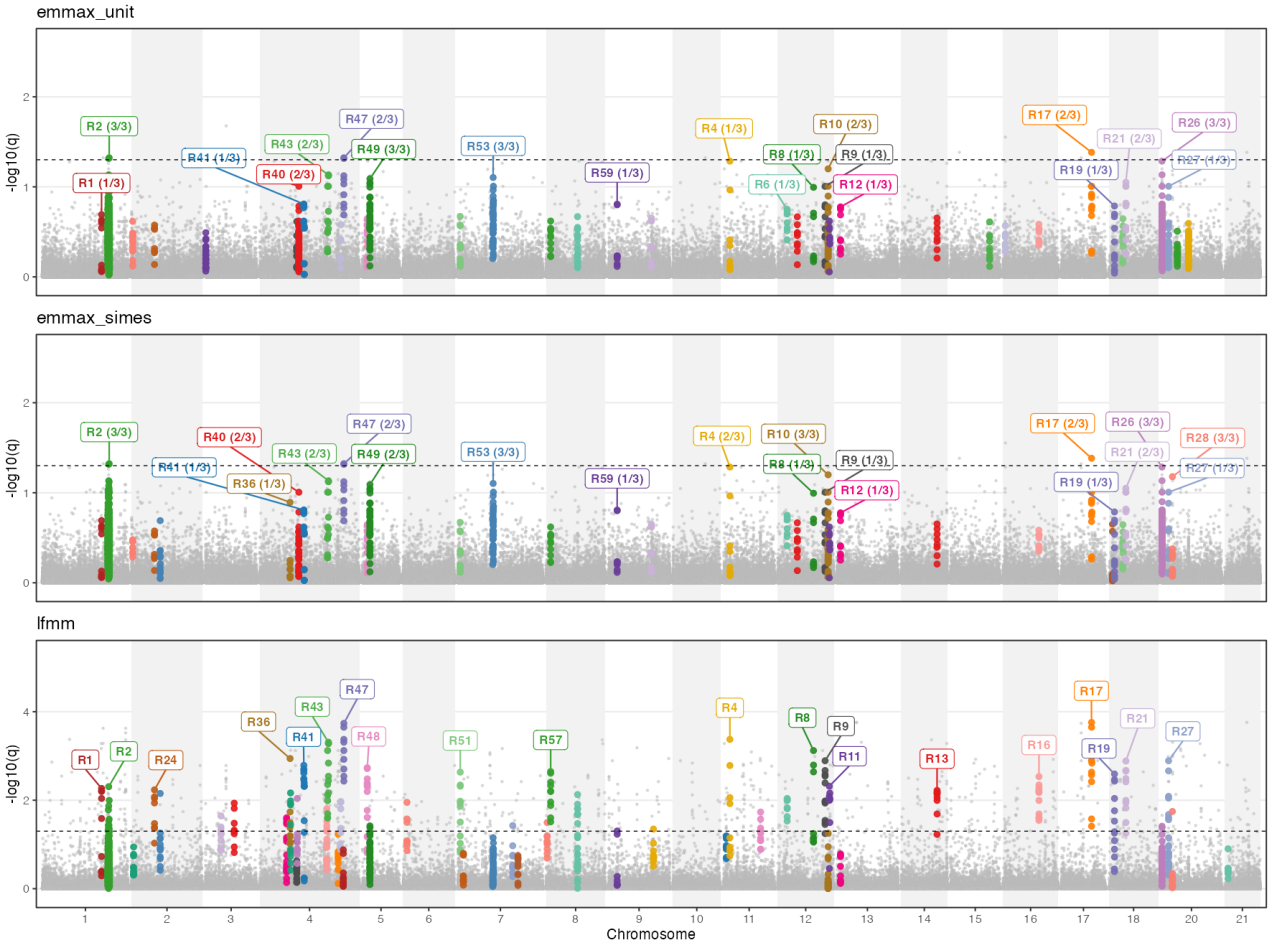


Figure 2: `output/figures/manhattan.png` for the full three-spined panel (20 chromosomes), with $-\log_{10}(q)$ on the vertical axis and one panel per engine (consensus, Simes and LFMM2). The two EMMAX panels show the floor-stability tiers.

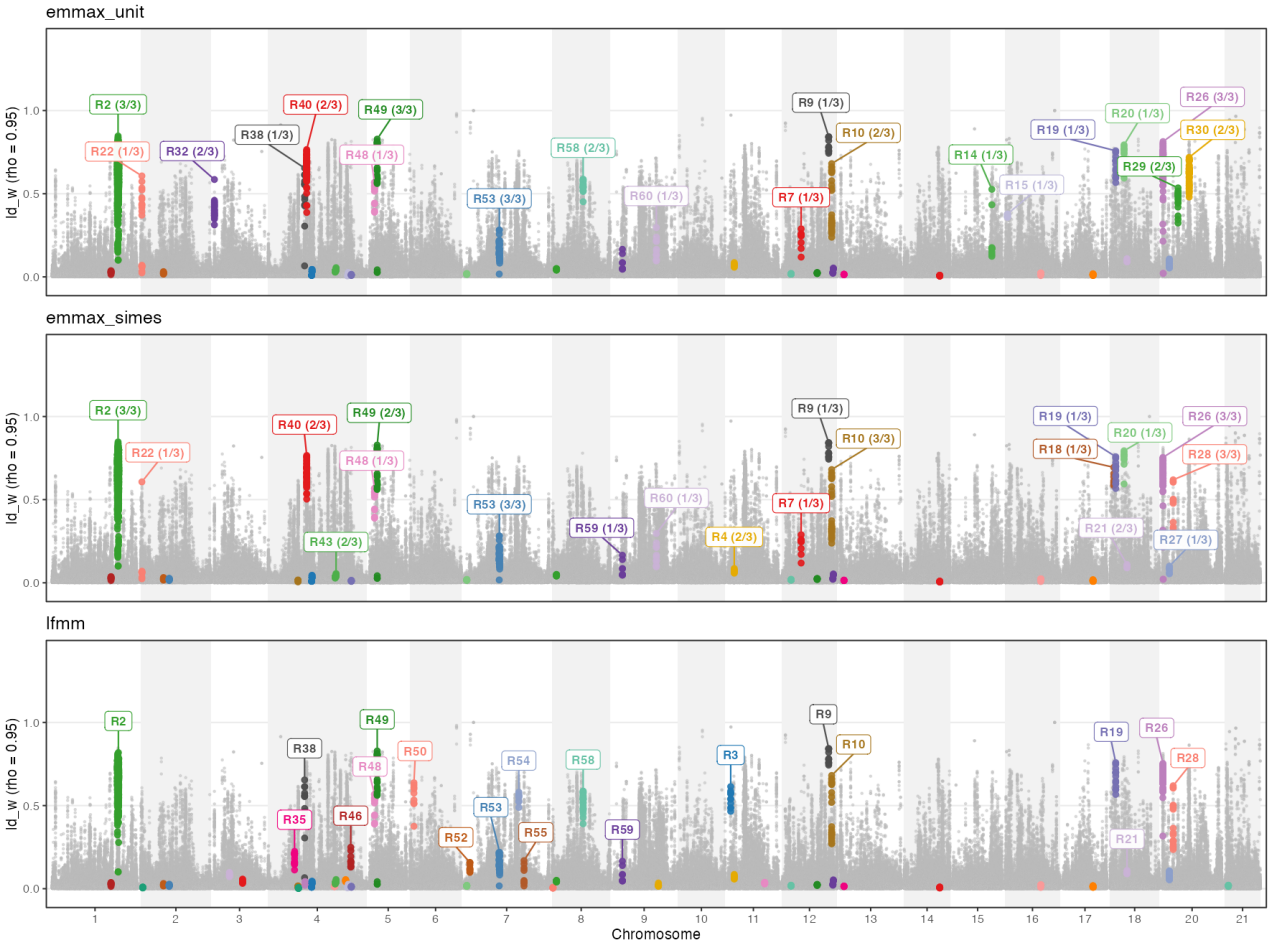


Figure 3: The same data set and panels, with `ld_w_095` on the vertical axis instead of $-\log_{10}(q)$.

8 Batch summary: `run_all()`

Function: `run_all(dataset_dirs)`. It runs `run_dataset()` on every folder. A failing data set is recorded with `status = "error"`, and the others still run. It writes two summary files to the working directory, and `run_batch.R` writes them to the repository root.

`summary.csv`: one row per data set and engine

Column	Meaning
`dataset`	`basename(dataset_dir)`.
`engine`	The p-value engine (`emmax_unit`, `emmax_simes` or an external engine). `NA` if the data set had no engine or failed before reaching one.
`n_markers`	The number of markers in the data set's map.
`n_units_tested`, `n_significant`, `n_regions`	Units tested, significant units and outlier regions for this engine at the reported floor.
`runtime_s`	Wall-clock seconds for the whole `run_dataset()` call. It is the same on every row of a data set.
`status`	`"ok"`, `"no_engine"` (the data set ran but had no p-values to test) or `"error"`.
`error`	The error message when `status = "error"`, and `NA` otherwise.
`align_grm_axes_r2`, `align_grm_axes_p`, `align_structure_r2`, `align_structure_p`	The two main structure-alignment measures and their tail proportions. They are repeated on every row of a data set, so that many data sets can be sorted on association results and on the risk of confounding at the same time. All are `NA` without `input/population.rds`.

The four `align_*` columns use the within-group permutation null, the manuscript's primary null. `align_grm_axes_r2` and `align_structure_r2` are the observed values, and `align_grm_axes_p` and `align_structure_p` are the corresponding `proportion_null_ge_observed`. Low values flag a phenotype whose signal is hard to separate from ancestry or sampling structure.

`alignment_summary.csv` gives details for each data set with an alignment result (requiring both population and phenotype inputs). It holds `dataset`, every column of `observed.csv` and three columns for each scheme and measure in `null_summary.csv`. Characters other than letters and digits in the scheme name become `_`. The columns are named:

```
<scheme>__<metric>__null_mean
<scheme>__<metric>__observed_within_null95
<scheme>__<metric>__proportion_null_ge_observed
```

9 Configuration reference

`resolve_config(dataset_dir)` starts from `DEFAULTS` in `R/00_config.R` and applies overrides from `input/config.R`. A supplied `cfg` is a complete configuration, not a partial list of changes. To override settings for one call, resolve the configuration first:

```
cfg <- resolve_config("examples/3sp_chr1_chr4")
cfg$cr_rho <- 0.35
run_dataset("examples/3sp_chr1_chr4", cfg = cfg)
```

Stage 1 and LD decay

Setting and default	Meaning
`cr_rho` Default: 0.5	Relative LD setting used to derive chromosome-specific cutoffs for Stage-1 clustering (`ld_complexity_reduction()`).
`decay_args$slide` Default: 500	Window size, in markers, for the LD-decay fit and Stage-1 clustering. Larger windows require more memory and time.
`decay_args$rho_targets` Default: 0.99	The share of LD for which the LD-decay report gives recommended windows. It does not change the fit.
`decay_args` (other fields) Default: see `R/00_config.R`	The other LD-decay settings, such as the MAF filter, the quantile for background LD, the subsample sizes and the number of decay windows.

Association scan

Setting and default	Meaning
`statistics` Default: `c("unit","simes")`	Which tests of the association scan to run.
`unit_repr` Default: `"consensus_dosage"`	How a unit's markers are combined into one genotype for the consensus test.
`grm_method` Default: `"GCTA"`	The `SNPRelate::snpgdsGRM()` method used for the GRM.
`b_unit` Default: 1000	Permutations for the consensus test.
`b_simes` Default: 200	Permutations for the Simes test, which is more expensive per permutation because every eligible marker is rescanned.

Unit testing and region assembly

Setting and default	Meaning
`size_floor` Default: `NULL` (derived)	The floor (Section 2.4). `NULL` derives it from marker density, and a number sets it. It is never below 2.
`size_floor_markers_per_mb` Default: 250	Marker-density divisor used to derive the floor: $\max(2, \text{round}(\text{density}/250))$ at the default.
`alpha` Default: 0.05	Significance threshold after BH correction.
`assembly` Default: `"stage2_discovered"`	How significant units are assembled into outlier regions, either by a second clustering of the significant units (`"stage2_discovered"`) or by merging their spans (`"physical"`).
`score_threshold` Default: 0.80	LD threshold for `"stage2_discovered"` assembly.
`distance_threshold` Default: 10^5	The largest distance, in base pairs, across which `"stage2_discovered"` assembly merges units.
`gap` Default: 3×10^5	The largest gap, in base pairs, across which `"physical"` assembly merges units.

Relocation test and computing

Setting and default	Meaning
`n_rotations` Default: 10000	Number of relocations in the relocation test (`ld_region_rotation()`), used only with an `annotation`.
`rotation_scheme` Default: `"within"`	Relocation within chromosomes.
`seed` Default: 1	Random seed.
`cores` Default: 1	Parallel workers for the permutations. Each extra worker holds its own copy of the working data, so watch the memory with many markers. On a 36 GB laptop, four workers ran the full panels, whereas eight workers left almost no free memory.

10 Caching and receipts

Each analysis step (Stage 1, the association scan, unit testing for each engine and the structure-alignment check) saves a hash of its inputs and settings in a `_receipt.rds` file next to its output, and it skips the work when nothing has changed. The floor profile and the floor sweep are made of these steps, so they use the same receipts. The hashes are of file **contents**, not timestamps, so a `git checkout` or a copied file cannot hide a change in contents. Unchanged runs reuse cached analysis results, but still read input and cached files and check their hashes. The LD-decay report is rewritten, and plots are redrawn unless `plot = FALSE`. Pass `force = TRUE` to an analysis step to rebuild its results even when the receipt is current.

A receipt is also out of date when one of its recorded outputs is missing or when the validated `LDscnR` revision has changed. The receipt of the association scan also depends on the Stage-1 receipt, and unit testing records the contents of any p-values or annotation passed to it directly. Use `force = TRUE` only when you want to rebuild results although the inputs and settings are unchanged.

11 Reproducing the manuscript

`manuscript/` runs this workflow on the two full stickleback panels of the manuscript and compares every result with the manuscript's analysis modules (`module_3sp` and `module_9sp` in the LDscnR-paper repository). The default settings already match the modules' settings (Table 7). The nine-spined panel adds `covariate.rds` (lineage), and both panels permute populations within groups with `perm_within_groups()`.

Table 7: Settings shared by the manuscript and the defaults of `LDscnR-multi`.

Setting	Value
Stage-1 LD setting	`cr_rho = 0.50`
LD-decay window (`slide`)	500 markers
GRM	GCTA, from the representative marker of every Stage-1 unit
Floor	marker-density rule (8 for three-spined, 12 for nine-spined)
Permutations	1,000 (consensus) and 200 (Simes)
EcoPeak relocation test	10,000 relocations within chromosomes
Floor profile	half and twice the floor, recovery as in Section 7.4
Random seed	1

The scripts need the modules' Stage-1 bundles and data from a local LDscnR-paper checkout (environment variable `LDSCNR_PAPER`). Without them, `manuscript/` documents the reproduction but cannot rerun it.

```
Rscript manuscript/build_inputs.R    # data set folders from the module bundles
Rscript manuscript/run.R             # all steps, including the floor profile
Rscript manuscript/compare.R --out=manuscript/comparison_vs_v091
```

`manuscript/run.R` calls the workflow's steps directly, without the Manhattan plots and the structure-alignment check. For a new data set, the entry point is `run_dataset()` or `run_batch.R`. For the three-spined panel, `run.R` also runs the EcoPeak relocation test on every engine's outlier regions and on each floor-stability tier. EcoPeaks are the marine-freshwater differentiated regions of Kingman et al. (2021).

`manuscript/comparison_vs_v091.md` lists every check. With LDscnR v0.9.1, all 108 checks match the modules' results: the Stage-1 units, the GRM, the LD-decay fit, every unit p-value, the significant units, the outlier-region spans, the permutation counts, the EcoPeak relocation tests, the floor-stability tiers and the EcoPeak tests per tier. `manuscript/comparison_vs_stored.md` compares with the modules' earlier outputs from LDscnR commits before v0.9.1. It matches except for the EcoPeak relocation null mean, fold and p of the three three-spined tests (9 of 108 checks), which changed because v0.9.1 keeps relocated regions inside their chromosome.

The LFMM2 p-values are not computed here. `manuscript/build_inputs.R` converts the manuscript's stored LFMM2 F statistics (`LEA::lfmm2.test()`, $K = 7$) to p-values with `pf(F, 1, n - 2)`, exactly as the modules do. The stored F statistics are already divided by the genomic inflation factor (their median equals the null median exactly), so these p-values are genomic-control corrected.

12 Known limitations

- **Plotting at genome scale.** `ldm_manhattan()` plots every marker. Figures 2 and 3 show that this works for a full panel of 790,578 markers, but a much larger panel would benefit from thinning or rasterising the points.
- **One data set folder per genotype panel and phenotype.** Screening many phenotypes on the same genotype panel needs one folder per phenotype. Each folder then rebuilds the same Stage 1 and stores its own copy of `genotypes.rds` and `map.rds`. This is fine when the data sets are different genomic panels, but it is costly for many phenotypes on one panel, and sharing Stage 1 between phenotypes would need a change of design.
- **Scope.** Unit testing accepts p-values from any method, but the workflow runs only `LDscnR`'s own outlier analysis. It has no interface for other outlier methods, such as OutFLANK, BayeScan or pcadapt.

13 How to cite

If you use `LDscnR-multi`, please cite the manuscript and the `LDscnR` package. `CITATION.cff` in the repository holds the same information in machine-readable form. `LDscnR-multi` is distributed under the MIT licence (`LICENSE`), the same licence as `LDscnR`.

- Kemppainen P, Guillaume F (2026). LDscnR: LD-aware complexity reduction for genome scans. *bioRxiv*. https://doi.org/10.64898/2026.10.27.762116. Submitted to *Molecular Ecology Resources*.

- Kemppainen P. LDscnR, version 0.9.1. R package. https://github.com/genomeinsights/LDscnR.

The example data and the reproduction use:

- Fang B, Kemppainen P, Momigliano P, Merilä J (2021). Population structure limits parallel evolution in sticklebacks. *Molecular Biology and Evolution* 38: 4205–4221. https://doi.org/10.1093/molbev/msab144. Genotype data: https://doi.org/10.5281/zenodo.4722879 (CC BY 4.0). The example genotypes are two-chromosome subsets of these data.
- Kingman GAR et al. (2021). Predicting future from past: the genomic basis of recurrent and rapid stickleback evolution. *Science Advances* 7: eabg5285. https://doi.org/10.1126/sciadv.abg5285. EcoPeaks: FigShare project 162634 (https://figshare.com/projects/kingsleyAssemblyHub/162634), used only by the reproduction and not distributed here.

Methods referred to in this manual:

- Kang HM et al. (2010). Variance component model to account for sample structure in genome-wide association studies. *Nature Genetics* 42: 348–354.
- Caye K, Jumentier B, Lepeule J, François O (2019). LFMM 2: fast and accurate inference of gene-environment associations in genome-wide studies. *Molecular Biology and Evolution* 36: 852–860.
- Yang J, Lee SH, Goddard ME, Visscher PM (2011). GCTA: a tool for genome-wide complex trait analysis. *American Journal of Human Genetics* 88: 76–82.

A Full output directory tree

```
<dataset_dir>/
  input/
    genotypes.rds, map.rds, phenotype.rds, perm_fun.R,
    population.rds, structure_group.rds, covariate.rds, config.R
  external_pvalues/
    <engine>.rds                          # one per extra p-value engine
  cache/                                  # regenerable
    genotypes.gds, stage1.rds, _receipt.rds
  output/                                 # regenerable
    ld_decay/
      decay_summary.csv, decay_print.txt, decay_curves.pdf
    alignment/
      observed.csv, null_draws.csv, null_summary.csv, _receipt.rds
    pvalues/emmax_unit/
      p_obs.rds, p_perm.rds, p_obs_marker.rds, params.rds, _receipt.rds
    pvalues/emmax_simes/
      p_obs.rds, p_perm_compact.rds, params.rds, _receipt.rds
    floor_profile/
      grid.csv, summary.csv,
      canonical_regions_unit.csv, canonical_regions_simes.csv
    stageB_<engine>/
      outlier_test.rds, outlier_perm.rds, region_rotation.rds,
      snp_results.csv, region_table.csv, report.txt, _receipt.rds
    figures/
      manhattan.png, manhattan.pdf,
      manhattan_ld_w.png, manhattan_ld_w.pdf
    floor_sweep_summary.csv               # only after run_floor_sweep()
    figures/manhattan_floor_sweep.{png,pdf}

<working directory or repository root>/
  summary.csv, alignment_summary.csv      # only after run_all()
```